Shuttle Dispatch

Value and Policy Iteration

The infinite-horizon solution is to **wait when fewer than 13 passengers are waiting and dispatch when 13 or more are waiting**. The sections below specify the model, explain each solution method, and present numerical evidence that the implementation is correct.

1 The model: what happens in one period?

Each period has four steps: observe the queue, choose whether to dispatch, charge the dispatch and waiting costs, and then add the new arrivals. The state records only the queue length because the assumed arrival distribution and available actions do not otherwise depend on the past.

Table 1: Notation used to describe the environment.

Symbol	Meaning
t	Decision period; the first decision is at $t = 0$.
s_t	Number of passengers waiting *before* the decision at time t. Possible states are $0, 1, \ldots, 200$.
a_t	Action: 0 means wait; 1 means dispatch one shuttle.
N_t	Random number of new arrivals after the decision. Each of $1, 2, 3, 4, 5$ has probability $1/5$. Arrivals are independent across periods.
$c(s, a)$	Cost of taking action a when the queue contains s passengers.
γ	Discount factor, 0.95. A cost one period later receives weight 0.95 relative to a cost now.

A shuttle can serve 15 passengers. Therefore, after action a in state s, the number still waiting is $\max\{s - 15a, 0\}$. The maximum with zero prevents the queue from becoming negative. The cost is

$$\underbrace{c(s,a)}_{\text{current cost}} = \underbrace{100a}_{\text{dispatch cost}} + \underbrace{2\max\{s-15a,0\}}_{\text{cost of passengers left waiting}} . \tag{1}$$

New arrivals are added after this cost is assessed. The next queue is

$$s_{t+1} = \min\{\max\{s_t - 15a_t, 0\} + N_t,\ 200\}\,. \tag{2}$$

The minimum with 200 enforces the station capacity.

Example. With 20 passengers, waiting costs $2(20) = 40$ and leaves 20 passengers before arrivals. Dispatching serves 15, leaves 5, and costs $100 + 2(5) = 110$. If three passengers then arrive, the next queue is 23 after waiting or 8 after dispatch. Dispatching costs more *now*, but may save enough future waiting cost to be optimal.

Additional assumptions. Only one shuttle can depart per period, with no separate fleet or travel-time restriction. New arrivals first incur waiting cost at the next decision. Overflow above 200 is discarded without an extra penalty. These timing and overflow choices are assumptions, rather than consequences of the queue cap alone. Ties between actions are resolved by waiting.

2 Finite-horizon enumeration: 500 decisions

2.1 The quantity being optimized

Let $J_t(s)$ be the **minimum expected discounted cost still to be paid**, starting with s passengers at time t. Costs are discounted relative to that time. With 500 decisions, its definition is

$$J_t(s) = \min_{\pi} \mathbb{E}_{\pi}\left[\sum_{u=t}^{499} \gamma^{u-t} c(s_u, a_u) \,\middle|\, s_t = s\right]. \tag{3}$$

Here u counts future periods; π denotes a policy, meaning a rule for choosing actions; and $\mathbb{E}_{\pi}$ means an average over arrival randomness when that policy is followed. A smaller $J_t(s)$ is better. The report uses *cost values*; if rewards were defined as negative costs, their values would have the opposite sign.

2.2 Backward induction over states and actions

Part (a)'s "enumeration" is interpreted as evaluating every state, both actions, and all five possible arrivals at each period. There is no terminal penalty, so $J_{500}(s) = 0$. Starting at the last decision and working backward, compute

$$Q_t(s,a) = c(s,a) + \frac{\gamma}{5}\sum_{k=1}^{5} J_{t+1}(\min\{\max\{s-15a, 0\} + k, 200\}), \tag{4}$$

$$J_t(s) = \min_{a\in\{0,1\}} Q_t(s,a). \tag{5}$$

$Q_t(s,a)$ is the expected remaining cost if action a is taken *now* and decisions are optimal afterward. In the sum, k is a possible arrival count. The factor $1/5$ averages the five outcomes, and γ discounts their continuation costs. The minimizing action is saved as $\pi_t(s)$, the decision rule at time t.

This procedure reuses previously computed continuation values, avoiding the exponential cost of listing every complete decision history. The implementation saves all 501 value vectors and all 500 policy vectors.

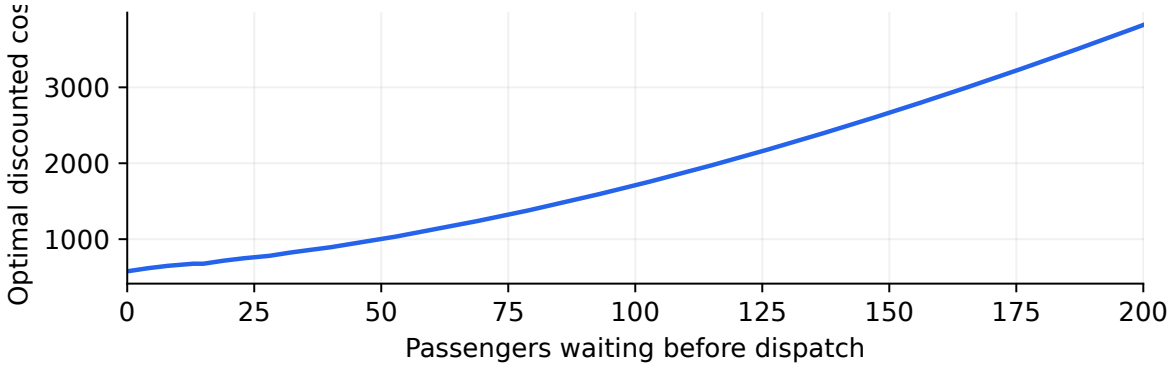


Figure 1: Minimum expected discounted cost at the initial decision, $J_0(s)$, for 500 decisions and zero terminal cost.

The finite-horizon policy need not be stationary: it can change as the end approaches. In the final period, waiting is optimal in every state. A dispatch costs 100 but saves at most $2 \times 15 = 30$ immediately; there is no later benefit or terminal penalty to justify it.

3 Infinite-horizon value iteration

For an unlimited sequence of decisions, write $J^*(s)$ for the minimum expected discounted cost starting from queue s. The superscript $*$ indicates optimality. This value has no time subscript because the problem has no deadline and the environment does not change over time.

3.1 The Bellman update

Starting from $J^{(0)}(s) = 0$, repeatedly compute

$$J^{(n+1)}(s) = \min_{a\in\{0,1\}} \left[c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J^{(n)}(\min\{\max\{s-15a,0\}+k, 200\}) \right]. \tag{6}$$

The superscript (n) counts algorithm iterations, not decision periods. Each update asks: *which action has the lowest current cost plus discounted expected future cost?* Its structure is the same as the finite-horizon recursion, but it improves an estimate of the stationary value instead of moving backward through time.

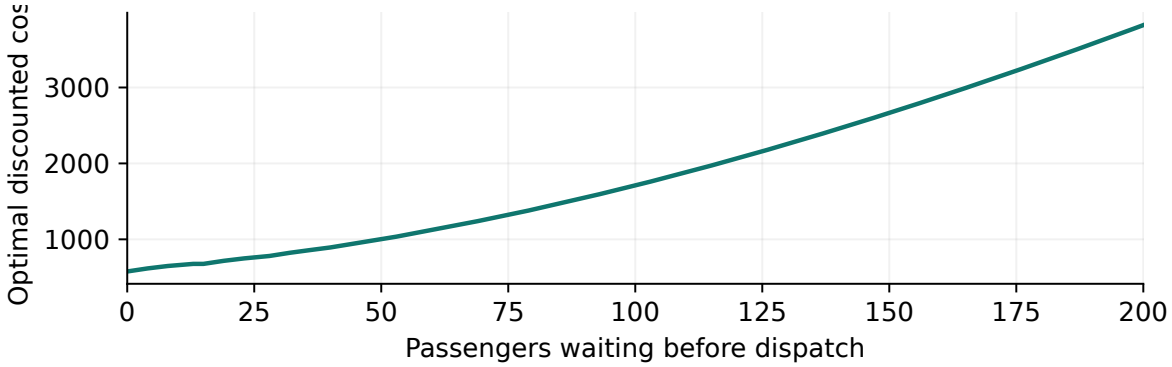


Figure 2: Optimal infinite-horizon cost $J^*(s)$, estimated by value iteration. Larger initial queues incur greater total expected cost.

3.2 When does iteration stop?

Let $\mathcal{T}J$ denote the entire right-hand side of Equation (6), evaluated using a candidate value vector J. This is the *Bellman operator*: one application performs one value update. Define its residual as

$$\delta(J) = \max_{s\in\{0,\ldots,200\}} |(\mathcal{T}J)(s) - J(s)|. \tag{7}$$

The residual measures how much another Bellman update would change the candidate values, in the worst state. The algorithm stops when $\delta(J) \le 10^{-10}$, which required **518 updates**.

Because $\gamma < 1$, the Bellman operator contracts maximum errors by at most γ. Consequently,

$$\max_s |J(s) - J^*(s)| \le \frac{\delta(J)}{1-\gamma}. \tag{8}$$

This connects a computable stopping criterion to the accuracy of the value estimate.

The implementation minimizes costs. Thus, its arrays named `finite_costs`, `vi_cost`, and `pi_cost` are the cost values J_t and J^* described above.

4 Policy iteration and the dispatch rule

Policy iteration alternates between evaluating a fixed decision rule and improving it. It starts from the rule "always wait."

Policy evaluation. For a stationary policy π, $\pi(s)$ is its action in state s. Let J_π be the vector of expected discounted costs when that rule is followed indefinitely. Let P_π be the 201×201 transition matrix: entry (s, s') is the probability of next queue s' after taking action $\pi(s)$. Let c_π be the vector with entries $c(s, \pi(s))$. Then

$$J_\pi = c_\pi + \gamma P_\pi J_\pi, \qquad \text{so} \qquad (I - \gamma P_\pi) J_\pi = c_\pi. \tag{9}$$

Here I is the identity matrix, and s' denotes a possible next state. The linear system computes the cost of the current policy. When several arrival counts produce the same next state, their probabilities are added in P_π.

Policy improvement. For every state, compare both actions using J_π as the continuation value in Equation (6). Replace the current action with the lower-cost one. Stop when no actions change. This took **six evaluation/improvement rounds**, including the final stability check.

The resulting optimal stationary rule is

$$\boxed{\pi^*(s) = \begin{cases} 0 & \text{(wait)}, \quad 0 \le s \le 12, \\ 1 & \text{(dispatch)}, \quad 13 \le s \le 200. \end{cases}} \tag{10}$$

The threshold was discovered by comparing both actions in every state; it was not imposed on the search. Departing with 13 or 14 passengers can be worthwhile because the savings in future holding costs outweigh the benefit of waiting for a full shuttle.

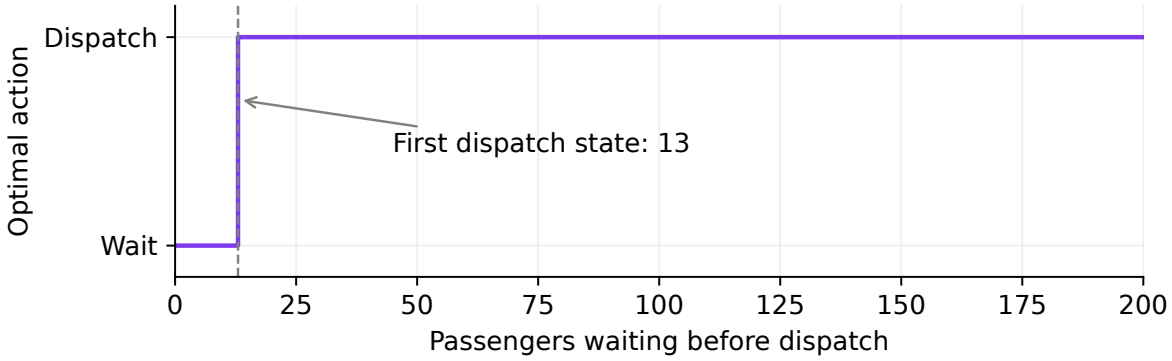


Figure 3: Optimal stationary policy from policy iteration. The first dispatch state is 13.

Table 2: Representative minimum infinite-horizon discounted costs.

Initial passengers s	Optimal cost $J^*(s)$
0	576.670168
15	676.670168
200	3821.586477

The difference $J^*(15) - J^*(0) = 100$ is a useful consistency check. Dispatch at 15 clears the queue, while waiting at zero also leaves it empty. Both lead to the same next-arrival distribution; only the dispatch charge differs.

5 Edge cases and correctness checks

All **14 validation checks passed**. The checks use optimality conditions, independent solution procedures, and model changes with predictable outcomes.

5.1 Agreement and optimality

Value iteration and policy iteration selected identical actions in all 201 states. Their maximum value difference was 1.964×10^{-9}. The policy-iteration result had Bellman residual 9.095×10^{-13}. Equation (8) therefore bounds its maximum value error by approximately 1.819×10^{-11}, subject to floating-point rounding. The smallest absolute difference between the two action costs was 3.778, well above the numerical error.

The maximum difference between the 500-period time-zero values and infinite-horizon values was 4.942×10^{-9}. Since the largest possible stage cost is 470, a conservative bound on the discounted cost beyond 500 periods is

$$\sum_{t=500}^{\infty} \gamma^t \, 470 = \frac{470\gamma^{500}}{1-\gamma} = 6.838 \times 10^{-8}. \tag{11}$$

The observed difference is smaller than this bound. Agreement is expected because very late costs receive little weight under discounting.

5.2 Queue boundaries and the final period

Service never produces a negative queue: 15 passengers clear exactly, and 16 leave one. At state 200, waiting always gives next state 200; dispatch gives next states 186 through 190. At zero, dispatch costs 100 more than waiting without changing the next-state distribution, so it is strictly dominated. These cases verify that service, saturation, and probability pooling are implemented consistently.

The terminal value vector was checked to be zero. The last-period policy was also checked against the analytic prediction that waiting is optimal everywhere. This tests that the finite-horizon algorithm does not charge continuation costs beyond its deadline.

Table 3: Model modifications with known answers. In this table, $J(s)$ denotes the optimal cost for the modified problem.

Modification	Predicted result	Observed result
Zero holding cost	Always wait; $J(s) = 0$.	Matched all states.
Zero shuttle capacity and no arrivals	Always wait; $J(s) = 2s/(1-\gamma)$.	Zero analytic error.
Free dispatch, capacity 200, no arrivals	Dispatch at nonempty states; $J(s) = 0$.	Matched all states.
Zero discount factor	Only current costs matter: always wait, with $J(s) = 2s$.	Matched all states.

A separately coded exhaustive decision/arrival tree for a seven-state, four-period example matched backward induction exactly. Complete enumeration is feasible at this scale and provides an independent check of the finite-horizon recursion.

6 Independent simulation and reproducibility

The optimal policy was simulated from initial queues of 0, 15 and 200. Each starting state used 20,000 independent trajectories of 300 decisions, with random seed 1668. A trajectory's total cost is the sum of its stage costs weighted by $1, \gamma, \gamma^2, \ldots$. The simulator calculated transitions and costs directly instead of reusing the solver's transition-array builder.

Table 4: Simulation estimates compared with exact policy evaluation.

Initial queue	Simulation mean	Exact cost	Standard error
0	576.671469	576.670168	0.219213
15	676.814669	676.670168	0.221520
200	3821.605190	3821.586477	0.532280

The standard error estimates uncertainty in the simulation *mean*: it is the sample standard deviation of trajectory costs divided by $\sqrt{20{,}000}$. Each mean was within four standard errors of the exact value, allowing an additional 0.001951 for the conservative bound on costs omitted after period 300. This wider tolerance is the automated simulation check; Figure 4 displays approximate 95% intervals using ± 1.96 standard errors.

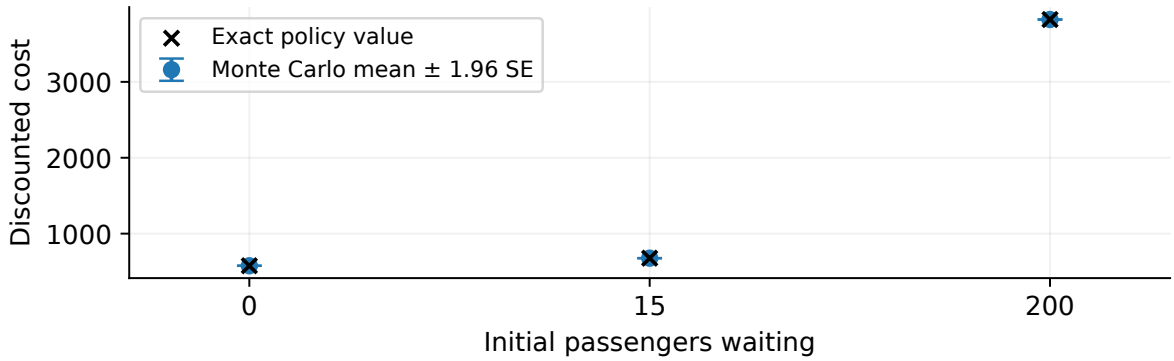


Figure 4: Independent simulation means and exact policy costs. Error bars describe uncertainty in the mean, not the variation of individual trajectory costs.

Simulation supports the computed policy values but does not prove optimality. The Bellman residual provides the stronger optimality check. Likewise, algorithm agreement alone cannot catch every modeling mistake because the methods share the same specification; the analytic examples and independently coded tree and simulator provide additional evidence.

6.1 Reproducing the results

From the project folder, run:

```
python -m pip install -r requirements.txt
python shuttle.py
jupyter lab shuttle_plots.ipynb
```

The solver runs all three methods and all validation checks, raising an error if any check fails. Full evidence is saved in `results/summary.json`; per-state results are in `results/state_results.csv`. The saved arrays in `results/shuttle_results.npz` include every finite-horizon value and policy. Running all notebook cells regenerates the assignment plots in `results/figures/`. For the compact vector figures used in this report, run `make_report_figures.py`; then compile `SHUTTLE_REPORT.tex`.

7 Validation with an additional three months

An extended study would focus on three questions.

How sensitive is the dispatch rule to the parameters?

Vary dispatch and holding costs, shuttle capacity, discounting, and arrival distributions. Record changes in the optimal threshold and test whether threshold behavior persists. Compare larger queue caps, such as 250, 500 and 1,000, and introduce penalties for lost arrivals. These experiments would reveal whether the 13-passenger recommendation depends strongly on the chosen parameters or overflow assumption.

Can another method confirm the solution?

Construct an independent linear-programming formulation and compare its values and actions with dynamic programming. Use higher precision when action costs are nearly tied. Vary the finite horizon and terminal penalties to study decisions near the deadline. Use more simulation seeds and starting queues, and measure how often nominal confidence intervals contain the exact policy value.

Does the policy remain effective under more realistic conditions?

Replace independent uniform demand with bursty or correlated arrivals, and evaluate the original policy under these alternative demand processes. Add shuttle availability and travel times to the state. These extensions would test robustness to modeling assumptions, beyond verifying numerical correctness for the stated environment.

Assignment 1: Policy Gradient, Parts 1 and 2

GitHub: FailedMath3U **Repository:** MLDS
`https://github.com/FailedMath3U/MLDS`

1 Implementation

Both tasks use PyTorch Monte Carlo policy gradients and complete episodes. CartPole uses `CartPole-v1`; Pong uses `ALE/Pong-v5`, the namespaced Pong-v5 environment. Actions are sampled during training and evaluation.

For decision t, s_t is the observed input, a_t the action, r_t its subsequent reward, and T the episode length. The policy $\pi_\theta(a \mid s)$ is a neural network with parameters θ. With discount γ, the objective and reward-to-go are

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

For a batch of B episodes, Adam minimizes

$$\mathcal{L} = -\frac{c}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i-1} \gamma^t \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})(G_t^{(i)} - b_t^{(i)}).$$

Here i indexes episodes, b_t is the baseline (zero in Part 1), and c is a fixed scale. Returns and baselines are detached from actor gradients; no return centering or standardization is used. Plots show *undiscounted* episode rewards, as requested.

Setting	CartPole	Pong
Input	4 physical observations	6,400 frame-difference pixels
Hidden layer; output	64 tanh; 2 logits	200 ReLU; 2 logits
Discount γ; scale c	0.95; 0.05	0.99; 1
Actor Adam learning rate	0.01	0.001
Episodes per update; total	32; 8,000	8; 20,000
Gradient-norm cap	10	5
Training seeds	1668, 1669, 1670	1668
Hardware	CPU	One GPU

Pong uses only actions 2 and 3 (RIGHT and LEFT), with no startup FIRE/NOOP. Frames are cropped and downsampled to 80×80, backgrounds 144/109 removed, and foreground binarized using the assignment's preprocessing. Consecutive binary frames are subtracted to supply motion; the reset input is zero. Default frame skip 4 and sticky-action probability 0.25 are retained. Full games are episodes: returns never reset at points. Gradients accumulate over all eight games before one update.

Pong's alternative rate 0.001 gives a smaller update: on identical game data, mean policy KL divergence after one update was 1.996×10^{-6}, versus 3.423×10^{-4} at 0.01. This motivates a conservative rate but does not establish better long-run stability. Clipping affected 61.68% of basic Pong updates and 18.64% with a baseline; CartPole never reached its cap.

Evaluation. Each main result uses the final fixed-budget checkpoint and 500 fresh episodes, without learning. Test reset seeds are separate from training and shared across methods. CartPole seed 1668 was selected before training for the main plots; all three seeds are reported below. SD denotes population SD across the 500 episode rewards.

2 Part 1(a): CartPole-v1

The final primary policy scored **500.000 ± 0.000**. All 500 evaluations reached the 500-step limit. The untrained neural policy scored 20.926 ± 11.467 and a uniform random policy 22.782 ± 12.058 over 500 episodes each. Training improved performance substantially, but included large temporary declines.

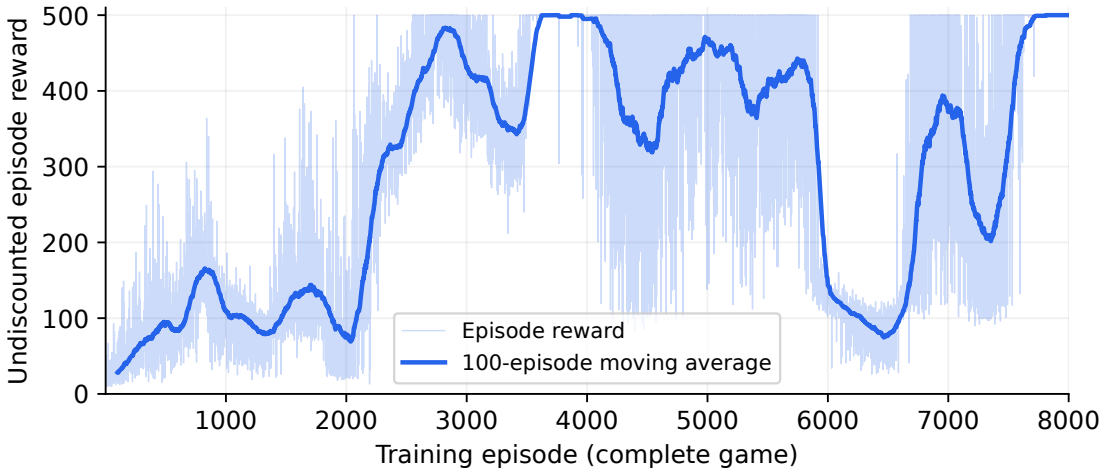


Figure 1: CartPole without a baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 episodes.

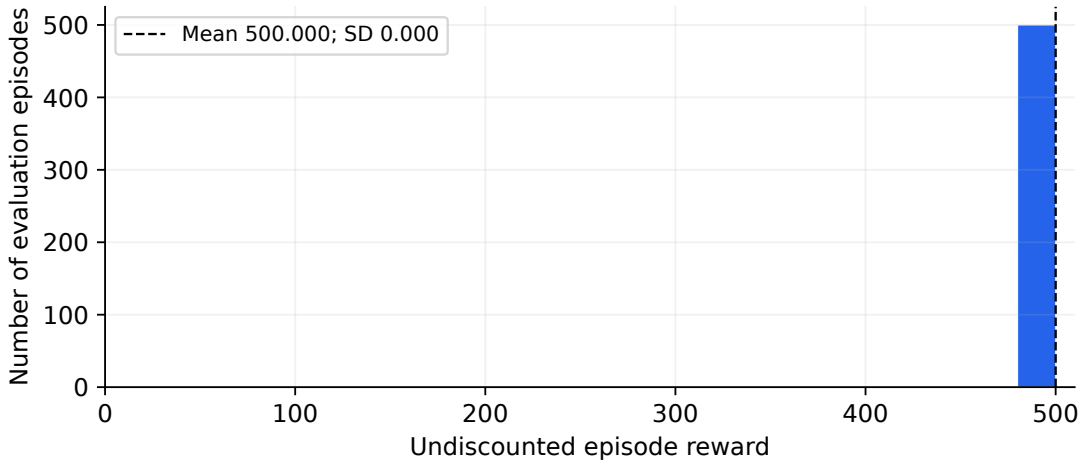


Figure 2: CartPole without a baseline: 500 final-policy evaluation rewards. Mean 500.000; population standard deviation 0.000.

3 Part 1(b): Pong-v5

The final policy scored **-18.610 ± 1.681**, ranging from −21 to −10. All 500 games were losses; none was truncated. An untrained reference scored −20.250 ± 0.935 over 32 games. Performance improved modestly, but **the policy did not learn to win**.

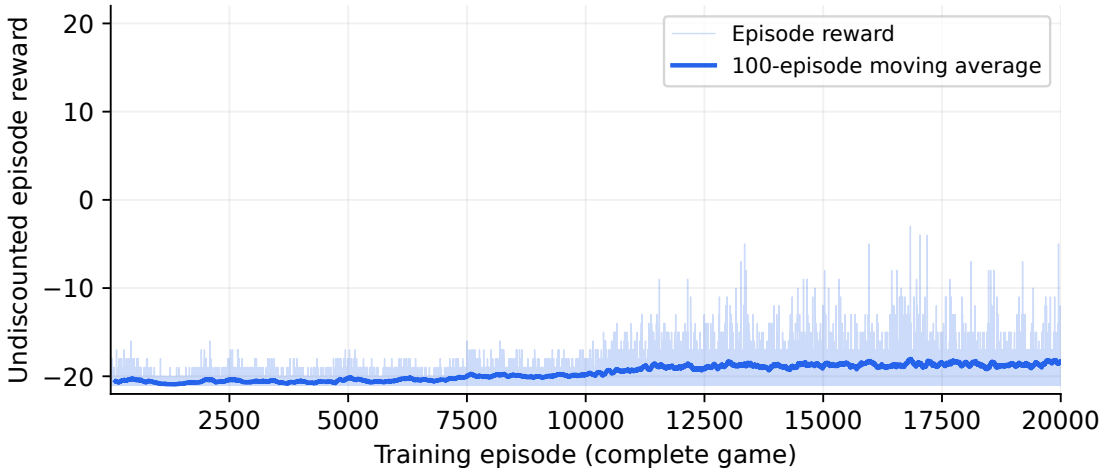


Figure 3: Pong without a baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.

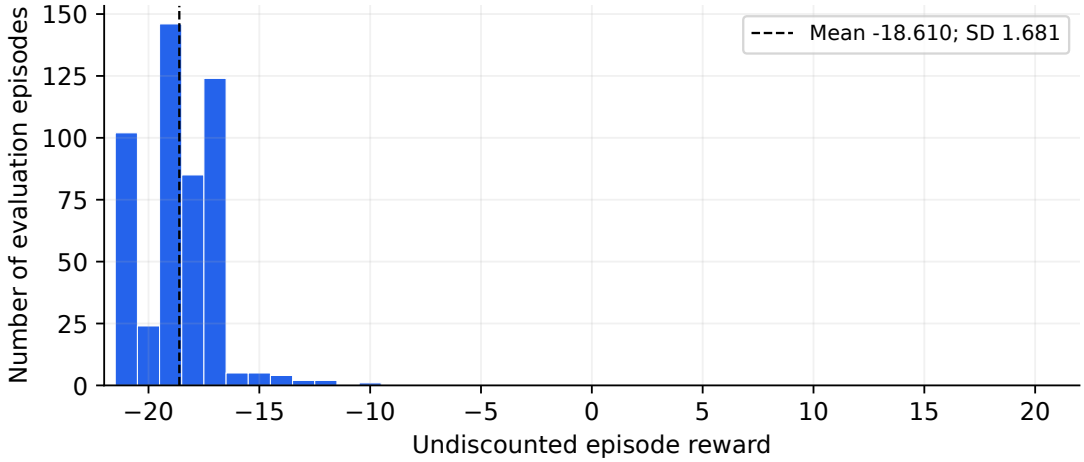


Figure 4: Pong without a baseline: 500 final-policy evaluation rewards. Mean -18.610; population standard deviation 1.681.

4 Part 2(a): CartPole with a baseline

The baseline is a separate network with inputs $(s_t, t/500)$, 64 tanh units and one output predicting G_t. Predictions use weights from earlier batches and are frozen for the actor update. Afterward, five Adam steps at rate 0.01 fit Monte Carlo returns by mean squared error. Actor settings and initial parameters match Part 1.

The final model scored **260.308 ± 57.013**. Four episodes reached 500; 496 ended in physical failure. This primary baseline run performed worse than the basic method.

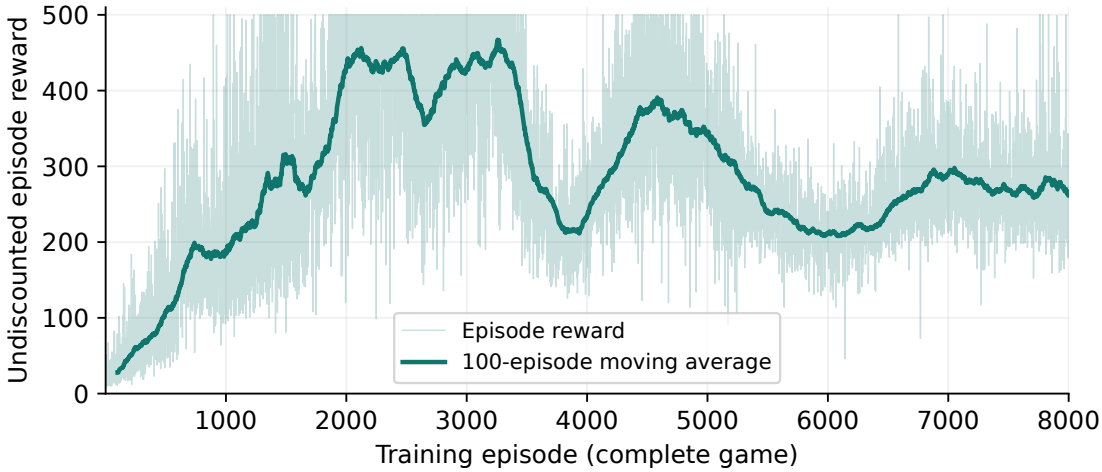


Figure 5: CartPole with a learned baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 episodes.

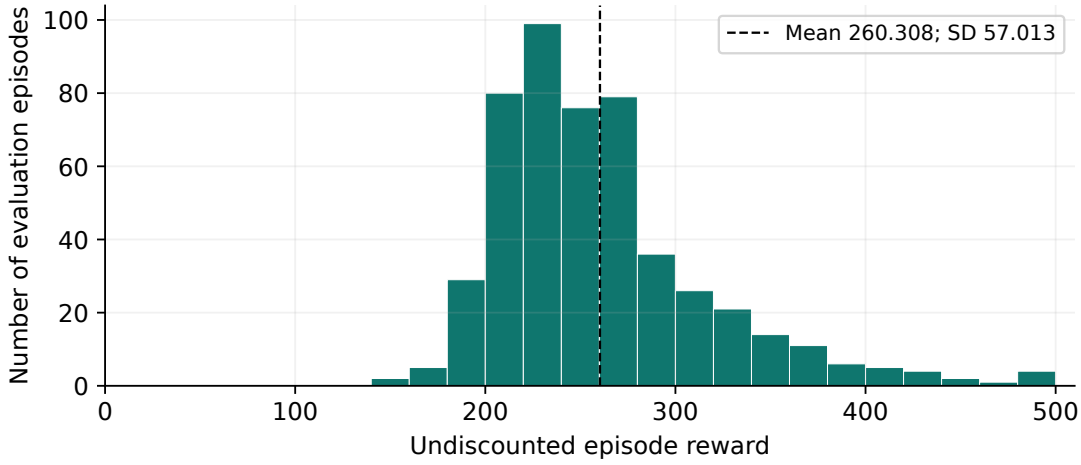


Figure 6: CartPole with a learned baseline: 500 final-policy evaluation rewards. Mean 260.308; population standard deviation 57.013.

5 Part 2(b): Pong with a baseline

The baseline is a lagged scalar exponential moving average. Update k uses b_k, starting at $b_0 = 0$; afterward,

$$\bar{G}_k = \frac{\sum_{i,t} \gamma^t G_t^{(i)}}{\sum_{i,t} \gamma^t}, \qquad b_{k+1} = 0.95 b_k + 0.05 \bar{G}_k.$$

Thus the baseline uses only previous batches when forming an actor gradient. An action-independent baseline preserves the expected unclipped gradient because $\mathbb{E}_a[b \nabla_\theta \log \pi_\theta(a \mid s)] = b \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$. Here ∇_θ denotes differentiation with respect to policy parameters. Clipping can alter this expectation.

The final score was **-18.446 ± 1.851**, only 0.164 above Part 1. All 500 games were losses (range −21 to −5), with no truncations. One training seed cannot establish a general benefit.

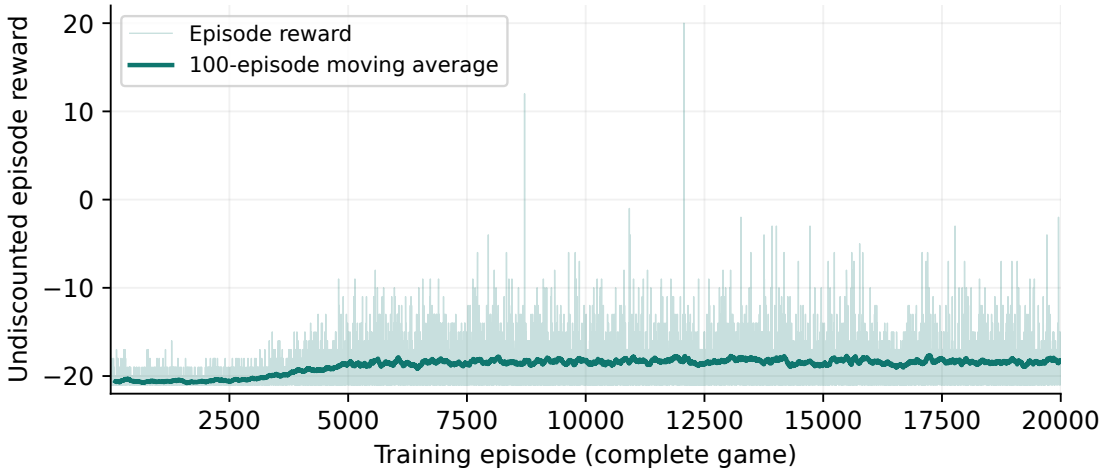


Figure 7: Pong with an EMA baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.



Figure 8: Pong with an EMA baseline: 500 final-policy evaluation rewards. Mean -18.446; population standard deviation 1.851.

6 Additional experiments and interpretation

Training seeds and learning rate

Method	Seed	Mean	SD	Fraction at 500
No baseline	1668	500.000	0.000	1.000
No baseline	1669	271.824	28.856	0.000
No baseline	1670	483.886	65.298	0.928
Learned baseline	1668	260.308	57.013	0.008
Learned baseline	1669	360.516	79.141	0.100
Learned baseline	1670	349.940	76.321	0.092

Across the three seeds, mean evaluation reward was 418.570 without a baseline and 323.588 with it. The baseline improved seed 1669 but not the others.

Further runs changed only the actor rate, retaining the same three seeds, budgets and 500-episode evaluations; the critic rate stayed 0.01:

Actor rate	No baseline: mean across seeds	Learned baseline: mean across seeds
0.01	418.570	323.588
0.005	163.178	357.661
0.002	247.102	264.481

Smaller rates did not consistently improve final performance. These are exploratory comparisons across three seeds, not evidence of an optimal rate.

Were CartPole's dips just episode noise?

Both original seed-1668 runs were replayed exactly: all 8,000 rewards and final parameter hashes matched. Frozen checkpoints before and during selected dips were then evaluated on 500 fresh paired reset/action seeds. Basic-policy means fell from 453.108 (episode 5,120) to 83.114 (6,400): paired change -369.994, bootstrap 95% interval $[-377.430, -362.038]$. Baseline-policy means fell from 440.466 (3,072) to 215.686 (6,144): change -224.780, interval $[-231.886, -217.436]$.

Holding the earlier basic policy fixed for 2,000 fresh episodes gave 100-episode averages of 430.11–472.29; continued training reached 74.64. The baseline's frozen range was 413.00–453.82, versus a continued-training minimum of 211.37. Policy quality changed beyond ordinary fixed-policy episode noise. Noisy updates may cause such changes, but this experiment does not identify the optimization mechanism. Checkpoints were selected after inspecting training curves; intervals are conditional on these policies, not uncertainty across training seeds.

What the discounted objective rewards

CartPole's discounted return is $(1 - 0.95^T)/0.05$, approaching 20 even for episodes shorter than 500. The primary baseline model averaged 19.999743 despite its lower undiscounted score. In Pong, discounted evaluation return improved from -1.309180 at initialization to 0.313913 (basic) and 0.293346 (baseline), while full-game scores remained poor. At $\gamma = 0.99$, rewards 1,000 decisions away receive weight 4.32×10^{-5}. This suggests a credit-assignment limitation, but is not a causal test.

7 Edge cases and quantitative validation

CartPole: both methods

Returns and gradient. Rewards $(1, 1, 1)$ at $\gamma = 0.95$ must give returns $(2.8525, 1.95, 1)$; the code matches. Setting $\gamma = 0$ gives immediate rewards. In an enumerable two-step task with binary-action reward and action probability $p = \text{sigmoid}(\theta)$, the analytic gradient is $(1 + \gamma)p(1 - p)$. Enumeration gave 0.471188756 versus analytic 0.471188748, with and without a constant baseline. A rewarded-action update increased its probability from 0.500000 to 0.502500, checking the loss sign.

End conditions. Forced cart-position and pole-angle violations terminated the episode and retained the terminal +1 reward. A one-step time limit truncated without physical failure. Either flag ends return accumulation, with zero continuation. Fixed reset seeds and ten fixed actions reproduced identical states. The saved ten-decision trace records states, action probabilities, actions and rewards for inspection.

Baseline mechanism. At a frozen initial actor, a baseline fitted on 512 separate episodes was evaluated on 512 paired held-out trajectories. Summed parameter-gradient variances fell from 5.338262 to 0.842421 (84.22%). The empirical mean-gradient difference was 0.0474, versus paired standard-error magnitude 0.1176. This supports variance reduction at that actor; it does not validate sustained reduction by the critic's five-step online fitting schedule. The controlled replay experiment above separately tests whether training dips reflect real policy changes.

Pong: both methods

Episode boundaries and actions. Rewards $(0, 1, 0, -1)$ at $\gamma = 0.99$ must yield

$$(G_0, G_1, G_2, G_3) = (0.019701, 0.0199, -0.99, -1).$$

The implementation matched within 10^{-6}. This checks that returns cross scoring boundaries. Zero discount recovered immediate rewards; the two-step analytic-gradient test agreed within 10^{-7}. Zero returns produced zero basic-policy gradient. Complete games reached scoring events and termination using only actions 2/3; workers reject other actions.

Images and replay. Known synthetic pixels verified crop, downsampling, background removal and foreground preservation without modifying the source image. Real inputs were zero at reset and otherwise in $\{-1, 0, 1\}$. Resetting frame history prevents leakage across games. Identical emulator seeds and fixed actions reproduced frame/reward hashes with sticky actions retained. Saved traces include the first ten decisions, a scoring event and the ending, with corresponding image differences.

Baseline mechanism. A separately fitted initial constant reduced summed gradient variance from 369.320 to 20.763 on 16 held-out games (94.38%). This candidate is distinct from the training EMA, which starts at zero. At 4,096, 8,192, 12,288 and 16,384 games, the actual EMA's paired variance ratios (with/without baseline, at the same frozen actor) were 0.873, 1.017, 0.924 and 1.011. It did not consistently reduce variance. These small 16-game probes have no variance-ratio confidence intervals and measure unclipped gradients.

Recovery and resources. Restoring actor and Adam state and repeating the next fixed-data update reproduced parameter hashes exactly. Preflight passed all 15 checks and verified one visible GPU. Both methods used that GPU sequentially with eight CPU emulator workers; training and evaluation completed in 2.64 hours. These checks support implementation correctness, while the game scores establish that learning remained inadequate.

8 Run instructions

Paths below are relative to the repository's `assignment1/` directory. Use a fresh copy for training to preserve saved results. Python 3.12, PyTorch 2.14.0, NumPy 2.5.3 and Gymnasium 1.2.2 were used; Pong also used ALE 0.11.2.

CartPole: install requirements, train all six models, evaluate each for 500 episodes, then run the numerical and environment tests:

```
cd mlds_assignment1_cartpole
python -m pip install -r requirements.txt
python cartpole.py suite --episodes 8000 --lr 0.01
python validate.py
jupyter lab cartpole_plots.ipynb
```

Validation evidence is in `validation/validation.json`; the notebook plots saved results without retraining. The controlled dip experiment is reproduced with `python instability_experiment.py`, followed by `python plot_instability_experiment.py`; see its saved `plan.json` and README for seeds and bootstrap details. The separate `cartpole_lr005` and `cartpole_lr002` folders each contain `run.sh`, which runs the six lower-rate experiments, validation and plotting.

Pong: in a fresh directory containing its source, config and requirements, activate a Python environment and install requirements. Select one GPU; set its UUID in `config.json` and `supervise.sh`. Then:

```
export CUDA_VISIBLE_DEVICES=<single-GPU-UUID>
export CUBLAS_WORKSPACE_CONFIG=:4096:8
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python run_pong.py preflight
nohup setsid bash supervise.sh > nohup.log 2>&1 < /dev/null &
```

Preflight executes the Pong checks above. Existing checkpoints resume rather than restart. See `preflight/results.json`, `runs/*/training.csv`, `runs/*/evaluation/test_final.json`, and `diagnostics/` for quantitative evidence. The source README documents recovery and traces.

Report: run `python build_assets.py`, then compile `POLICY_GRADIENT_REPORT.tex` with Tectonic or pdfLaTeX. Run `python combine_submission.py` to join the shuttle and policy-gradient PDFs.

9 Experiments with three additional months

CartPole: use more independent training seeds and separate tuning/test resets. Compare learning rates, batch sizes and baseline fitting schedules under matched budgets; measure critic error and paired gradient variance throughout training. Stress-test initial angles, velocities and altered physical parameters to assess generalization beyond standard resets.

Pong: keep v5, actions 2/3 and $\gamma = 0.99$ while comparing frame differences against frame stacks and a small convolutional policy. Measure gradient contributions by episode time to test delayed-credit limitations. Tune learning rate, clipping and batch size on validation seeds, then repeat training across seeds with untouched 500-game tests.

Baselines: compare scalar, state-dependent and time-dependent predictions at identical frozen actors, fitting on independent data. Use hundreds of paired trajectories and bootstrap variance-ratio intervals. Compare clipped and unclipped estimators to determine whether variance reduction translates into better control.